❷ Adds a log

This code outputs `-100` once and then stops the stream, logging the exception caused by dividing by zero:

```
-100
[...][ERROR][...] - [tried riskyHandler] Upstream failed.
java.lang.ArithmeticException: / by zero
      at examples.streams.two.SampleSpec.riskyHandler$1(SampleSpec.scala:247)
      ...
```

In this case, you can use `recover` to replace the exception with a default value that is generated instead:

```
class ExceptionsSpec ... {
  ...
         Source(-1 to 1)
          .map(riskyHandler)
          .log("tried riskyHandler")
          .recover {
            case e: ArithmeticException => 0     ❶
          }
          .run()
  ...
}
```

❶ Final element of the stream

This results in `0` when the `ArithmeticException` is thrown, giving the following output:

```
-100
0
[...][ERROR][...] - [tried riskyHandler] Upstream failed.
java.lang.ArithmeticException: / by zero
      at examples.streams.two.SampleSpec.riskyHandler$2(SampleSpec.scala:264)
       ...
```

But that's probably not what you want, and you have alternatives.